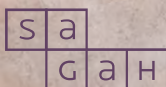


PROGRAMAÇÃO BACK END III

Pedro Henrique Chagas Freitas



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Sintaxe JSON

Objetivos de aprendizagem

- Descrever a importância do formato JSON.
- Identificar as diferenças básicas entre JSON e XML.
- Reconhecer a sintaxe do formato JSON.

Introdução

Neste capítulo, você vai estudar sobre a importância do formato JSON, sua sintaxe e as principais diferenças entre esse e a linguagem XML.

Importância do formato JSON

Para compreendermos o formato JSON, precisamos entender a finalidade para a qual ele foi criado. JSON é o acrônimo de *JavaScript Object Notation* e se trata de um subconjunto da linguagem JavaScript, porém não se limita a ela, apesar de usar a mesma sintaxe. Esse formato foi pensado para implementação de intercâmbio de dados. Obviamente, não temos somente o JSON, como, também, o XML (*eXtensible Markup Language*), que veremos mais à frente.

Como o XML veio antes, a ideia de intercâmbio de dados é anterior à criação do formato JSON. Logo, podemos dizer que XML é o pai do JSON, sendo que, este ganhou muito mais espaço por meio dos sistemas web, por trabalhar de forma mais simples, sem muita burocracia e já sendo integrado com o JavaScript, que é quase natural dentro de rotinas web.

A principal vantagem do JSON sobre o XML está no fato de o JSON ser mais leve e muito mais simples de ser lido e entendido (SILVA; SILVA JÚNIOR, 2018). Considere:

```
<?xml version= "1.0"encoding= "UTF-8"?>
<id>1</id>
<nome> João Maria</nome>
<endereco> Fortaleza </endereco>
```

Temos um exemplo de código XML. Já o mesmo código em JSON seria:

```
{"id":1,"nome":"João Maria","endereco":"Fortaleza"}
```

É possível perceber, nesse exemplo simples, por que o JSON é mais simples que o XML (TONSIG, 2008). Além disso, o JSON tem as seguintes características marcantes:

- possui *parsing* mais fácil;
- consegue suportar classes e objetos;
- é mais rápido na execução e no transporte de dados;
- cria arquivos menores;
- não é uma linguagem de marcação;
- não é um conjunto de esquemas de validação;
- é atualmente utilizado por Google, Facebook, Amazon, Microsoft, etc.

Por meio do JSON, por exemplo, podemos atribuir argumentos de forma simples a *strings* de nomes, como no boletim *on-line* de alunos de uma escola. Veja:

```
{"Aluno": [  
  {"nome": "Jorge Luiz Pereira", "notas": [9,5,10]}  
  {"nome": "Caio Carvalho Figueira", "notas": [3,6,9]}  
  {"nome": "Diego Rui Maria", "notas": [6,8,9]}  
]}
```

JSON *versus* XML

Conforme vimos, a melhor forma de comparar é por meio da visualização. Então, vamos ver como seria a definição de três jogadores de futebol de um time, utilizando três registros no XML e um vetor no JSON, respectivamente:

```
<Time>  
  <Jogador>  
    <primeiroNome>Cristiano</primeiroNome>  
    <ultimoNome>Ronaldo</ultimoNome>  
  </Jogador>  
  <Jogador>
```

```
<primeiroNome>Lionel</primeiroNome>
<ultimoNome>Messi</ultimoNome>
</Jogador>
<Jogador>
  <primeiroNome>Daniel</primeiroNome>
  <ultimoNome>Alves</ultimoNome>
</Jogador>
</Time>
```

Agora, vamos implementar o mesmo time que foi apresentado em XML, por meio de um vetor em JSON:

```
{
  "Time": [
    {"primeiroNome": "Cristiano", "ultimoNome": "Ronaldo"},
    {"primeiroNome": "Lionel", "ultimoNome": "Messi"},
    {"primeiroNome": "Daniel", "ultimoNome": "Alves"}
  ]
}
```

Respectivamente, podemos perceber que não existe limitação do uso do JSON em relação à utilização do XML — podemos usar tanto um como o outro. Em outras palavras, tudo que você faz com um pode ser feito com o outro. Porém, existem duas grandes diferenças entre XML e JSON.

A primeira é que, em XML, a análise sintática (ou *parsing*) deve ser feita necessariamente por um parser XML; o JSON, basta uma função JavaScript. A segunda é que o XML possui mecanismos para validação dos seus elementos (DTD e XSD), e não existe nada semelhante no JSON (SILVA; SILVA JÚNIOR, 2018).

Para compreendermos melhor, com XML, podemos escrever um DTD/XSD definindo que a *tag* <aluno> precisa obrigatoriamente de um <nome>, e o próprio *parser* sabe que existe um erro no XML, caso o elemento <nome> não exista. Há a validação dos elementos, além da forma. No JSON, só existe validação da forma, e caberá à aplicação validar as estruturas.

Pensem em um erro comum de caixa-alta, por exemplo. Ao invés de escrever o elemento <nome>, o desenvolvedor escreve <Nome> com “N” maiúsculo. No XML, o *parser* perceberia esse erro antes de passar para a aplicação, pois, apesar de estar com a forma correta, o elemento está errado de acordo com a sua definição no DTD/XSD.

No JSON, se a forma está correta, cabe à aplicação fazer as validações. Há, também, semelhanças: ambos são formatos texto (*plain text*), autodescritivos e facilmente entendíveis, hierárquicos e independentes de linguagens.

O JSON está constituído das seguintes estruturas:

1. uma coleção de pares nome/valor — em várias linguagens, isso é chamado de *object*, *record*, *struct*, dicionário, *hash table*, *keyed list*, ou *arrays* associativas (os identificadores em JSON devem estar definidos entre aspas, podendo ser simples ou duplas);
2. uma lista ordenada de valores — na maioria das linguagens, isso é caracterizado como uma *array*, vetor, lista ou sequência.

Essas são duas estruturas de dados universais. Virtualmente, todas as linguagens de programação modernas suportam-nas, de uma forma ou de outra (PRESSMAN, 2011). É aceitável que um formato de troca de dados que seja independente de linguagem de programação baseie-se nessas estruturas.

Sobre o XML, temos que é uma metalinguagem de marcação, especificada pelo World Wide Web Consortium (W3C), que define um conjunto de regras para codificar documentos em um formato que seja legível tanto por humanos quanto por máquinas. É importante destacar que o XML é capaz de prover o intercâmbio de documentos por meio da web, além de suportar uma grande gama de aplicações, permitindo a definição de elementos pelo usuário ou aplicação para estruturar o documento (SOMMERVILLE, 2008).

A fim de comparar as duas estruturas (JSON e XML), temos as diferenças básicas apresentadas no Quadro 1, a seguir.

Quadro 1. Diferenças básicas entre JSON e XML

XML	JSON
<i>Extensible Markup Language.</i>	<i>JavaScript Object Notation.</i>
Sintaxe similar à linguagem HTML.	Sintaxe similar à linguagem JavaScript.
Difícil de escrever o código.	Fácil de escrever o código.
Não consegue armazenar dados em variáveis.	Capacidade de armazenar até seis tipos de dados em variáveis.

(Continua)

(Continuação)

Quadro 1. Diferenças básicas entre JSON e XML

XML	JSON
É orientada a documento.	É orientada a objeto.
O dado tem maior segurança.	O dado tem menor segurança.
Arquivos menos legíveis.	Arquivos mais legíveis.
Exibição simplificada para o navegador Web descrever.	Não pode ser exibido no navegador sem análise.

Na Figura 1, a seguir, temos novamente a comparação do código em linguagem XML e em formato JSON.

XML

```
<empinfo>
  <employees>
    <employee>
      <name>James Kirk</name>
      <age>40</age>
    </employee>
    <employee>
      <name>Jean-Luc Picard</name>
      <age>45</age>
    </employee>
    <employee>
      <name>Wesley Crusher</name>
      <age>27</age>
    </employee>
  </employees>
</empinfo>
```

JSON

```
{ "empinfo" :
  {
    "employees" : [
      {
        "name" : "James Kirk",
        "age" : 40,
      },
      {
        "name" : "Jean-Luc Picard",
        "age" : 45,
      },
      {
        "name" : "Wesley Crusher",
        "age" : 27,
      }
    ]
  }
}
```

Figura 1. Comparação entre os códigos em linguagem XML e em formato JSON.

Sintaxe do formato JSON

Vamos ver como seria a definição da sintaxe do formato JSON para representar um *menu* de um restaurante com 10 pratos principais:

```
{
  "type": "Restaurante Menu",
  "restaurante-info": {
    "id": "121721",
    "sponsored": "0",
    "user-favourite": "",
    "name": "Restaurante Brasil",
    "address": "Rio de Janeiro, Copacabana, Q.10",
    "rating": "0.00",
    "minimum-order-price": "500.00",
    "delivery-fee": "100.00",
    "avg_delivery_time": "60",
    "open-status": "Aberto",
    "default-ad": "",
    "value": null,
    "total-category": "10",
    "site-url": "www.restaurantebrasil.com.br",
    "location-img": "",
    "restaurant-url": "/restaurantebrasil/121721/menu",
    "payment-methods": [
      {
        "id": "10",
        "type": "Cartão de débito"
      },
      {
        "id": "30",
        "type": "Cartão de crédito"
      },
      {
        "id": "40",
        "type": "Pagamento em dinheiro"
      }
    ],
    "delivery-types": "0",
```

```
"tags": [],
"Menu-Pratos": [
  {
    "id": "2",
    "name": "Macarrão com camarão internacional "
  },
  {
    "id": "3",
    "name": "      Peixe nordestino assado"
  },
  {
    "id": "4",
    "name": "Picanha gaúcha à moda da casa"
  },
  {
    "id": "5",
    "name": "Baião de dois à mineira"
  },
  {
    "id": "6",
    "name": " Farofa de banana da terra"
  },
  {
    "id": "7",
    "name": "Carne de sol do Cerrado"
  },
  {
    "id": "8",
    "name": "Tutu à mineira"
  },
  {
    "id": "9",
    "name": "Galinhada com pequi"
  },
  {
    "id": "10",
    "name": "Bife acebolado no arroz grego"
  }
]
```


Podemos concluir, então, que o JSON se trata de um subconjunto da linguagem JavaScript e que utiliza a mesma sintaxe. Hoje o JSON é empregado em diversas aplicações web, e a sua principal vantagem sobre o XML está no fato de ser mais leve e muito mais simples de ser lido e entendido, o que colabora diretamente para compreensão e implementação da sua sintaxe pelos.



Referências

PRESSMAN, R. S. *Engenharia de software: uma abordagem profissional*. 7. ed. Porto Alegre: AMGH; Bookman, 2011. 780 p.

SILVA, P. C.; SILVA JÚNIOR, J. B. Análise da representação semântica de modelos de dados do formato JSON. *Revista de Sistemas e Computação*, Salvador, v. 8, n. 1, p. 196–209, jan./jun. 2018. Disponível em: <https://revistas.unifacs.br/index.php/rsc/article/view/5487>. Acesso em: 26 nov. 2019.

SOMMERVILLE, I. *Engenharia de software*. 8. ed. São Paulo: Pearson Prentice Hall, 2008. 552 p.

Leitura recomendada

TONSIG, S. L. *Engenharia de software: análise e projeto de sistemas*. 2. ed. Rio de Janeiro: Ciência Moderna, 2008. 319 p.



Fique atento

Os *links* para *sites* da Web fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integridade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS